1 Design 0 Under Load Case 1

1.1 Envelope

The maximum Shear force occurs when the leftmost wheel is 1mm away from the start of the bridge (x=0mm). The maximum Bending Bending Moment occurs when the leftmost wheel is 128mm from the start of the bridge.

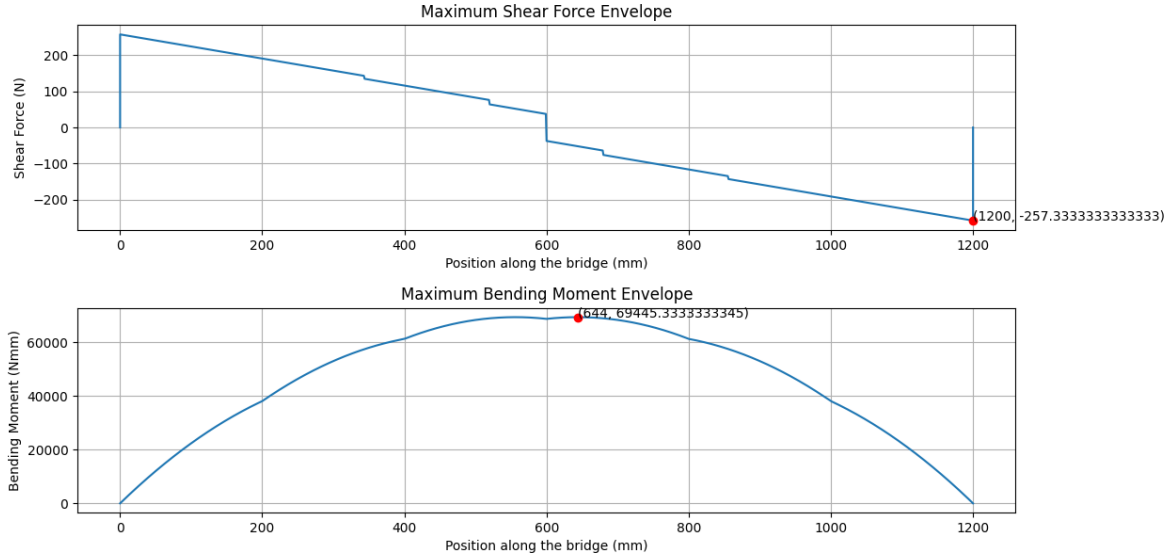

Figure 1: Envelope under Load Case 1

1.2 Intermediate calculations

Max Bending Moment: 69445.3333333345 $N \cdot m$
Max Shear Force: -257.3333333333333 N
The Centroid of the rectangles is: 41.43109435192319mm
The second moment of area of the rectangles is: 418352.20899942366 mm^4
the first moment of area of the rectangles at the centroid is: 6193.283330576373 mm^3
The first moment of area of the rectangles at glue is: 4343.896017305754 mm^3
Strength of buckling 1: 3.6848490831027525 MPa, Strength of buckling 2: 23.491114926631024 MPa, Strength of buckling 3: 29.430089518653464 MPa
Strength of shear buckling for each diaphragm: [4.923643246908379, 4.923643246908379, 4.923643246908379] MPa

1.3 FOS

$FOS_{tension}$: 4.362082243395873
$FOS_{compression}$: 1.0374943623234028
$FOS_{shearatplate}$: 2.6704331005630864
FOS_{glue}: 9.398467730570381
FOS_{buck_1}: 0.637168358288611
FOS_{buck_2}: 4.061983216845137
FOS_{buck_3}: 5.088925326379339

FOS_{buck_4}: 3.2870649754770116
Fail by type 1 local buckling.

1.4 Capacity

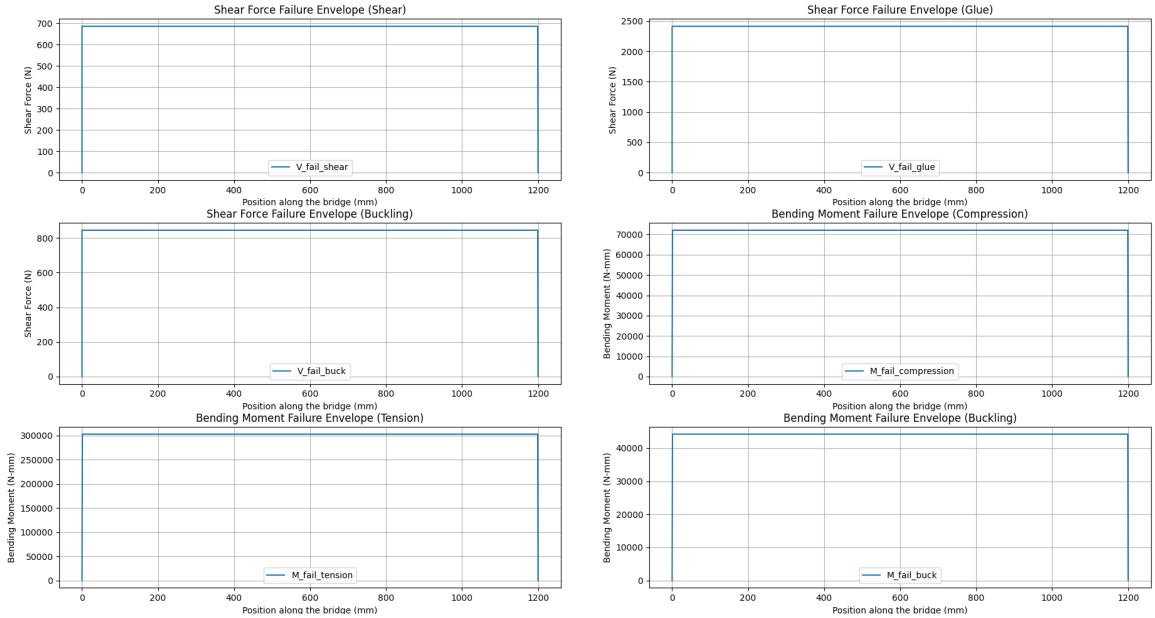

Figure 2: Capacity of Each type of Failure

2 Final Design Under Load Case 2

2.1 Envelope

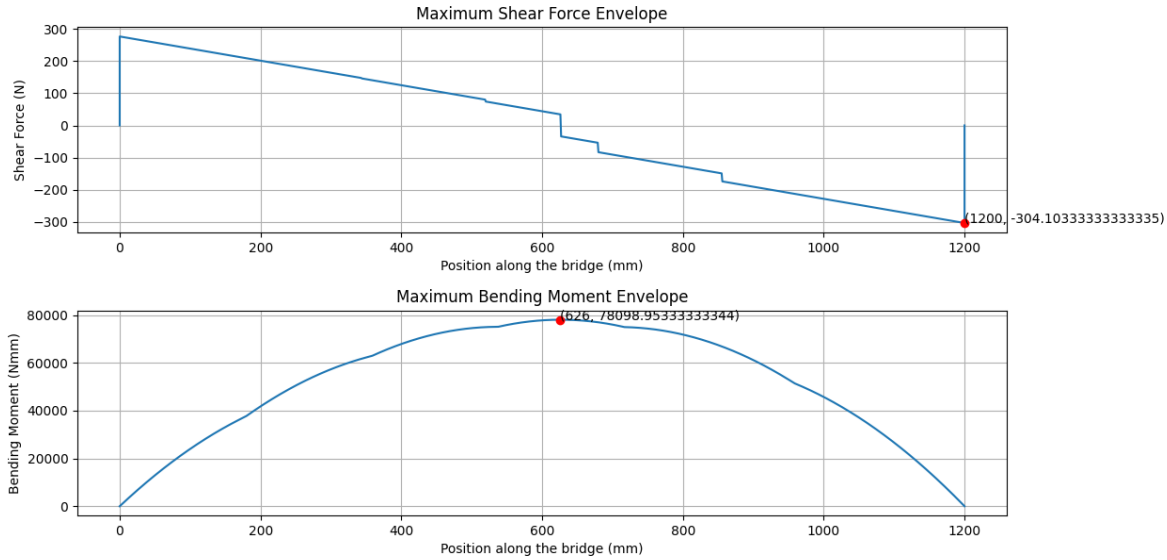

Figure 3: Envelope under Load Case 2. The maximum shear and bending moment are marked correspondingly.

2.2 Intermediate Calculation and FOS

```
Area of the component: 599.9226mm^2
Centroid of the rectangles is at: 74.25614695795758mm
Second moment of area of the rectangles is: 1.266122958799409 10e6 mm^4
first moment of area of the rectangles at centroid is: 12468.69186749907 mm^3
first moment of area of the rectangles at bottom glue is: 9125.020281506413 mm^3
first moment of area of the rectangles at top glue is: 5728.824336339387 mm^3
Strength of buckling 1: 25.87718675661888, Strength of buckling 2: 6.17780800174386, Strength of buckling 3: 17.76728176067848
Strength of shear buckling: [2.610123880736781, 2.610123880736781, 2.0919375220610967, 2.610123880736781, 2.610123880736781]
FOS_tension: 6.549671441602221, FOS_compression: 2.126420634689225, FOS_shear_plate: 3.396767139706596, FOS_glue: 18.766771088108985
FOS_buck_1: 9.170963981163522, FOS_buck_2: 2.189436402009392, FOS_buck_3: 6.2967857597073715, FOS_buck_4: 2.216495757162539
```

Figure 4: Output for Final Design under Load Case 2

2.3 Capacity

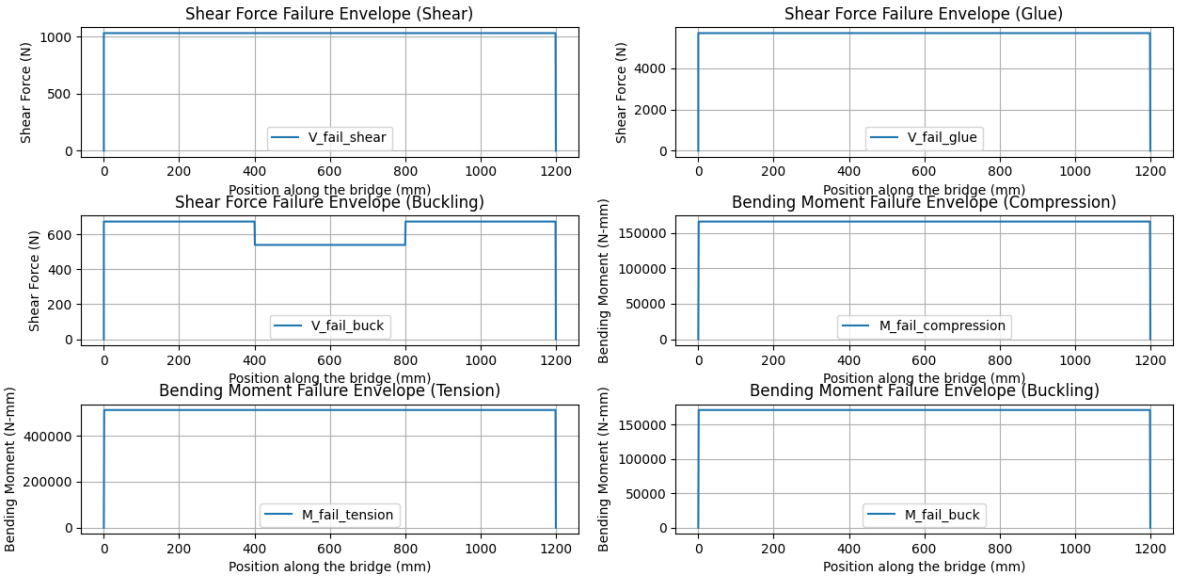

Figure 5: Capacity of the Final Design

3 Appendix A: Envelope

```
import numpy as np
import matplotlib.pyplot as plt

compressive_strength = 30
tensile_strength = 6
shear_strength_matboard = 4
young = 4000
poisson = 0.2
shear_strength_cemant = 2
x_train = [52, 228, 392, 568, 732, 908]
x_start = 0
x_end = 1200
#p_train = [182 / 2, 182 / 2,135 / 2, 135 / 2, 135 / 2, 135 /
    2] #Load Case 2
p_train = [400/6]*6 #Load Case 1

# Function to calculate shear force and bending moment at a
   given train position
def calculate_shear_force_and_bending_moment(train_position):
    real_x_train = [i + train_position for i in x_train]
    start_moment = sum(real_x_train[i] * p_train[i] for i in
       range(6))
    end_p = start_moment / 1200
    start_p = sum(p_train) - end_p
    shear_all=[0]*1201
    # Shear force calculation
    shear_force = [start_p]
    shear_positions = [0]
    for i in range(6):
        shear_force.append(shear_force[-1] - p_train[i])
        shear_positions.append(real_x_train[i])
    shear_force.append(0)
    shear_positions.append(1200)
    for pos in range(1201):
        if pos < real_x_train[0]:
            shear_all[pos] = start_p
        elif pos >= real_x_train[-1]:
            shear_all[pos] = -end_p
        else:
            for j in range(5):
                if real_x_train[j] <= pos < real_x_train[j +
                   1]:
                    shear_all[pos] = shear_force[j + 1]
                    break
    # Bending moment calculation
```

```
    bending_moment = [0]*1201
    for i in range(1,1201):
        bending_moment[i]=bending_moment[i-1]+shear_all[i-1]

    return shear_all, bending_moment

# Initialize envelope functions
max_shear_force = [0 for x in range(x_end + 1)]
max_bending_moment = [0 for x in range(x_end + 1)]

# Calculate envelope functions
for train_position in range(-52,293):
    shear_force, bending_moment =
       calculate_shear_force_and_bending_moment(
       train_position)
    for i in range(1201):
        max_bending_moment[i] = max(max_bending_moment[i],
           bending_moment[i])
    for i in range(1201):
        if abs(shear_force[i])>abs(max_shear_force[i]):
            max_shear_force[i] = shear_force[i]
    if abs(max(bending_moment)-69445.3333)<1e-3:
        print(train_position)
# Plotting envelope functions
plt.figure(figsize=(12, 6))
# Plotting maximum shear force envelope
x_pos=[i for i in range(x_end + 1)]
plt.subplot(2, 1, 1)
plt.plot([0]+x_pos+[1200], [0]+max_shear_force+[0])
plt.title('Maximum␣Shear␣Force␣Envelope')
plt.xlabel('Position␣along␣the␣bridge␣(mm)')
plt.ylabel('Shear␣Force␣(N)')
plt.grid(True)
# Plotting maximum bending moment envelope
plt.subplot(2, 1, 2)
plt.plot(range(x_end + 1), max_bending_moment)
plt.title('Maximum␣Bending␣Moment␣Envelope')
plt.xlabel('Position␣along␣the␣bridge␣(mm)')
plt.ylabel('Bending␣Moment␣(Nmm)')
plt.grid(True)
# Marking the highest shear force on the graph
max_shear_force_value = max(max_shear_force, key=abs)
max_shear_force_position = max_shear_force.index(
   max_shear_force_value)
plt.subplot(2, 1, 1)
plt.plot(max_shear_force_position, max_shear_force_value, 'ro
   ')
plt.text(max_shear_force_position, max_shear_force_value, f'
```

```
    ({max_shear_force_position}, {max_shear_force_value})')
# Marking the highest bending moment on the graph
max_bending_moment_value = max(max_bending_moment)
max_bending_moment_position = max_bending_moment.index(
    max_bending_moment_value)
plt.subplot(2, 1, 2)
plt.plot(max_bending_moment_position,
    max_bending_moment_value, 'ro')
plt.text(max_bending_moment_position,
    max_bending_moment_value, f'({max_bending_moment_position
    }, {max_bending_moment_value})')
plt.tight_layout()
plt.show()
```

4 Appendix B: FOS calculation and Capacity

```
import numpy as np
import matplotlib.pyplot as plt
import math

compressive_strength = 6
tensile_strength = 30
shear_strength_matboard = 4
young = 4000
poisson = 0.2
shear_strength_cemant = 2
x_train = [52, 228, 392, 568, 732, 908]
x_start = 0
x_end = 1200
#p_train = [135 / 2, 135 / 2, 135 / 2, 135 / 2,182 / 2, 182 /
    2]
p_train=[400/6]*6
M_fail_tension=[0 for x in range(1201)]
M_fail_compression=[0 for x in range(1201)]
M_fail_buck=[0 for x in range(1201)]
V_fail_shear=[0 for x in range(1201)]
V_fail_glue=[0 for x in range(1201)]
V_fail_buck=[0 for x in range(1201)]
# Function to calculate shear force and bending moment at a
   given train position
def calculate_shear_force_and_bending_moment(train_position):
    real_x_train = [i + train_position for i in x_train]
    start_moment = sum(real_x_train[i] * p_train[i] for i in
       range(6))
    end_p = start_moment / 1200
    start_p = sum(p_train) - end_p
    shear_all=[0]*1201
    # Shear force calculation
    shear_force = [start_p]
    shear_positions = [0]
    for i in range(6):
        shear_force.append(shear_force[-1] - p_train[i])
        shear_positions.append(real_x_train[i])
    shear_force.append(0)
    shear_positions.append(1200)
    for pos in range(1201):
        if pos < real_x_train[0]:
            shear_all[pos] = start_p
        elif pos >= real_x_train[-1]:
            shear_all[pos] = -end_p
        else:
            for j in range(5):
```

```
                if real_x_train[j] <= pos < real_x_train[j +
                    1]:
                    shear_all[pos] = shear_force[j + 1]
                    break
    # Bending moment calculation
    bending_moment = [0]*1201
    for i in range(1,1201):
        bending_moment[i]=bending_moment[i-1]+shear_all[i-1]

    return shear_all, bending_moment

# Initialize envelope functions
max_shear_force = [0 for x in range(x_end + 1)]
max_bending_moment = [0 for x in range(x_end + 1)]
# Calculate envelope functions
for train_position in range(-51,293):
    shear_force, bending_moment =
        calculate_shear_force_and_bending_moment(
        train_position)
    for i in range(1201):
        max_bending_moment[i] = max(max_bending_moment[i],
            bending_moment[i])
    for i in range(1201):
        if abs(shear_force[i])>abs(max_shear_force[i]):
            max_shear_force[i] = shear_force[i]
print(f"Max␣Bending␣Moment:␣{max(max_bending_moment)}",f"Max␣
    Shear␣Force:␣{min(max_shear_force)}" )


def local_buckling_stress(b, t, boundary_condition):
    k=0
    if boundary_condition == 'type␣1':
        k = 4.0
    elif boundary_condition == 'type␣2':
        k = 0.425
    elif boundary_condition == 'type␣3':
        k = 6
    sigma_cr = (k * (math.pi ** 2) * young) / (12 * (1 -
        poisson ** 2)) * ((t / b) ** 2)
    return sigma_cr
def shear_buckling_stress(a, h):
    tao_cr = (5* (math.pi**2) * young) / (12 * (1 - poisson
        ** 2))*((1.27/h)**2+(1.27/a)**2)
    return tao_cr

def first_moment_of_area(rectangles, h, verticel_glued_joints
    =None):
    centroid_y = centroid_of_rectangles(rectangles)
```

```
    first_moment = 0
    if verticel_glued_joints is None:
        for (x, y, width, height) in rectangles:
            area = width * height
            cy = y + height / 2
            if y + height <= h:
                first_moment += area * (centroid_y-cy)
            else:
                if y<h and y+height>h:
                    new_height=h-y
                    first_moment += width*new_height * (
                       centroid_y-(y+new_height / 2))
    else:
        for (x, y, width, height) in verticel_glued_joints:
            area = width * height
            cy = y + height / 2
            first_moment += area * (centroid_y-cy)
    return first_moment
def shear_stress_failure(rectangles,hori_glue,vert_glue,V,pos
    ):
    global FOS_shear_plate,FOS_glue, FOS_buck_4
    V=abs(V)
    I=second_moment_of_area(rectangles)
    #For the glue
    for (y,b) in hori_glue:
        Q=first_moment_of_area(rectangles,y)
        shear=V*Q/(I*b)
        FOS_glue=min(FOS_glue,shear_strength_cemant/shear)
        if V_fail_glue[pos]==0:
            V_fail_glue[pos]=shear_strength_cemant/shear*V
        else:
            V_fail_glue[pos]=min(V_fail_glue[pos],
               shear_strength_cemant/shear*V)
        if shear>shear_strength_cemant:
            print(f"Beam␣at␣{pos}mm␣in␣shear␣fail␣at␣glue")

    for (cross_sec,b) in vert_glue:
        Q=first_moment_of_area(rectangles,y,cross_sec)
        shear=V*Q/(I*b)
        FOS_glue=min(FOS_glue,shear_strength_cemant/shear)
        if V_fail_glue[pos]==0:
            V_fail_glue[pos]=shear_strength_cemant/shear*V
        else:
            V_fail_glue[pos]=min(V_fail_glue[pos],
               shear_strength_cemant/shear*V)
        if shear>shear_strength_cemant:
            print(f"Beam␣at␣{pos}mm␣in␣shear␣fail␣at␣glue")
    #Shear at Plate
```

```
    cy=centroid_of_rectangles(rectangles)
    Q=first_moment_of_area(rectangles,cy)
    b=1.27*2
    I=second_moment_of_area(rectangles)
    shear=V*Q/(I*b)
    FOS_shear_plate=min(FOS_shear_plate,
       shear_strength_matboard/shear)
    V_fail_shear[pos]=shear_strength_matboard/shear*V
    for i in range(len(tao_shear_buck)):
        if pos<=diaphram_pos[i+1] and pos>=diaphram_pos[i]:
            tao_cr=tao_shear_buck[i] #tao_cr at the position
            break
    FOS_buck_4=min(FOS_buck_4,tao_cr/shear)
    V_fail_buck[pos]=tao_cr/shear*V
    #if shear>shear_strength_matboard:
    #    print(f"Beam at {y}mm in shear fail at centroid")

def flexural_stress_failure(rectangles,pos):
    global FOS_tension,FOS_compression
    global strength_buck_1,strength_buck_2,strength_buck_3
    global FOS_buck_1,FOS_buck_2,FOS_buck_3
    I=second_moment_of_area(rectangles)
    cy=centroid_of_rectangles(rectangles)
    stress_at_top=abs(max_bending_moment[pos]*(
       height_of_bridge-cy)/I)
    stress_at_bottom=abs(max_bending_moment[pos]*(cy)/I)
    #print(f"Flexural Stress at the top: {stress_at_top}MPa",
       f"Flexural Stress at the bottom: {stress_at_bottom}MPa
       ")
    try:
        FOS_tension=min(FOS_tension,tensile_strength/
           stress_at_bottom)
        M_fail_tension[pos]=tensile_strength/stress_at_bottom
           *max_bending_moment[pos]
    except:
        FOS_tension=FOS_tension
    try:
        FOS_compression=min(FOS_compression,
           compressive_strength/stress_at_top)
        M_fail_compression[pos]=compressive_strength/
           stress_at_top*max_bending_moment[pos]
    except:
        FOS_compression=FOS_compression
    try:
        FOS_buck_1=min(FOS_buck_1,strength_buck_1/
           stress_at_top)
        FOS_buck_2=min(FOS_buck_2,strength_buck_2/
           stress_at_top)
```

```
            FOS_buck_3=min(FOS_buck_3,strength_buck_3/
               stress_at_top)
            M_fail_buck[pos]=min(strength_buck_1/stress_at_top*
               max_bending_moment[pos],strength_buck_2/
               stress_at_top*max_bending_moment[pos],
               strength_buck_3/stress_at_top*max_bending_moment[
               pos])

        except:
            FOS_buck_1=FOS_buck_1
            FOS_buck_2=FOS_buck_2
            FOS_buck_3=FOS_buck_3
        #if stress_at_top>compressive_strength:
        #    print(f"Top of the beam at {pos}mm in compression
           fail")
        #if stress_at_bottom>tensile_strength:
        #    print(f"Bottom of the beam at {pos}mm in tension
           fail")
    def centroid_of_rectangles(rectangles):
        """
        Parameters: rectangles (list of tuples): List of
           rectangles, each defined by (x, y, width, height).
        Returns: y coordinates of the centroid.
        """
        total_area = 0
        cy_total = 0

        for (x, y, width, height) in rectangles:
            area = width * height
            cy = y + height / 2
            total_area += area
            cy_total += cy * area

        centroid_y = cy_total / total_area

        return centroid_y
    def second_moment_of_area(rectangles):
        """
        Parameters: rectangles (list of tuples)
        Returns: Second moment of area (I).
        """
        I_total = 0
        centroid_y = centroid_of_rectangles(rectangles)

        for (x, y, width, height) in rectangles:
            area = width * height
            cy = y + height / 2
            dy = cy - centroid_y
```

```
            I_rect = (width * height**3) / 12
            I_total += I_rect + area * dy**2 #parallel axis
               theorem

    return I_total
def cross_section(component, position):
    """
    Parameters: Component, position (float)

    Returns: ist of rectangles at the given position.
    """
    cross_section_rectangles = []

    for (x, y, width, height, start_pos, end_pos) in
       component:
        if start_pos <= position <= end_pos:
            cross_section_rectangles.append((x, y, width,
               height))

    return cross_section_rectangles

def check_failure(component):
    for pos in range(0,1200):
        rectangle=cross_section(component,pos)
        flexural_stress_failure(rectangle,pos)
        cross_sec_hori_glue=[]
        cross_sec_vert_glue=[]
        for (height, thickness, start_pos, end_pos) in
           hori_glued_joints:
            if start_pos <= pos <= end_pos:
                cross_sec_hori_glue.append((height, thickness
                   ))
        for (cross_sec, thickness, start_pos, end_pos) in
           vert_glued_joints:
            if start_pos <= pos <= end_pos:
                cross_sec_vert_glue.append((cross_sec,
                   thickness))
        shear_stress_failure(rectangle,cross_sec_hori_glue,
           cross_sec_vert_glue,max_shear_force[pos],pos)

FOS_tension=1000
FOS_compression=1000
FOS_shear_plate=1000
FOS_glue=1000
FOS_buck_1=1000
FOS_buck_2=1000
FOS_buck_3=1000
FOS_buck_4=1000
```

```
#x_pos,y_pos,width,height,starting pos along the bridge,
   ending pos along the bridge
#Design 0 Parameter Start
component=[(10, 0, 80, 1.27,0,1200), (10, 1.27, 1.27,
   75-1.27,0,1200),
                (90-1.27, 1.27, 1.27, 75-1.27,0,1200)
                   ,(0,75,100,1.27,0,1200),
                (10+1.27,75-1.27,5,1.27,0,1200)
                   ,(90-1.27-5,75-1.27,5,1.27,0,1200)]
height_of_bridge=75+1.27
#Height, thickness of the connection (The program can
   determine Q at the height, so we don't need to hard code
   the components that are in connection),
# begining position along the bridge, ending position along
   the bridge
hori_glued_joints =[(75,6.27*2,0,1200)]
#Cross-sectional component, thickness of the connection,
   begining position along the bridge, ending position along
   the bridge
vert_glued_joints = []
diaphram_pos=[0,400,800,1200]
tao_shear_buck=[]
cy=centroid_of_rectangles(cross_section(component,0))
for i in range(len(diaphram_pos)-1):
    tao_shear_buck.append(shear_buckling_stress(diaphram_pos[
       i+1]-diaphram_pos[i],height_of_bridge))
strength_buck_1=local_buckling_stress(80-1.27*2,1.27,'type␣1'
   )
strength_buck_2=local_buckling_stress((100-80)/2,1.27,'type␣2
   ')
strength_buck_3=local_buckling_stress(75-cy,1.27,'type␣3')
#Design 0 Parameter end

#Final Design
#Final Design
check_failure(component)
print(f"Centroid␣of␣the␣rectangles␣is␣at:␣{cy}mm")
print(f"Second␣moment␣of␣area␣of␣the␣rectangles␣is:␣{
   second_moment_of_area(cross_section(component,0))/(10**6)}
   ␣10e6␣mm^4")
print(f"first␣moment␣of␣area␣of␣the␣rectangles␣at␣centroid␣is
   :␣{first_moment_of_area(cross_section(component,0),cy)}␣mm
   ^3")
print(f"first␣moment␣of␣area␣of␣the␣rectangles␣at␣glue␣is:␣{
   first_moment_of_area(cross_section(component,0),75)}␣mm^3"
   )
print(f"Strength␣of␣buckling␣1:␣{strength_buck_1},␣Strength␣
```

```
    of␣buckling␣2:␣{strength_buck_2},␣Strength␣of␣buckling␣3:␣
    {strength_buck_3}")
print(f"Strength␣of␣shear␣buckling:␣{tao_shear_buck}")
print(f"FOS_tension:␣{FOS_tension},␣FOS_compression:␣{
    FOS_compression},␣FOS_shear_plate:␣{FOS_shear_plate},␣
    FOS_glue:␣{FOS_glue}")
print(f"FOS_buck_1:␣{FOS_buck_1},␣FOS_buck_2:␣{FOS_buck_2},␣
    FOS_buck_3:␣{FOS_buck_3},␣FOS_buck_4:␣{FOS_buck_4}")


plt.figure(figsize=(12, 18))
x=list(range(1201))
# Plot V_fail_shear
plt.subplot(3, 2, 1)
plt.plot([0]+x, [0]+V_fail_shear, label='V_fail_shear')
plt.xlabel('Position␣along␣the␣bridge␣(mm)')
plt.ylabel('Shear␣Force␣(N)')
plt.title('Shear␣Force␣Failure␣Envelope␣(Shear)')
plt.legend()
plt.grid(True)

# Plot V_fail_glue
plt.subplot(3, 2, 2)
plt.plot([0]+x, [0]+V_fail_glue, label='V_fail_glue')
plt.xlabel('Position␣along␣the␣bridge␣(mm)')
plt.ylabel('Shear␣Force␣(N)')
plt.title('Shear␣Force␣Failure␣Envelope␣(Glue)')
plt.legend()
plt.grid(True)

# Plot V_fail_buck
plt.subplot(3, 2, 3)
plt.plot([0]+x, [0]+V_fail_buck, label='V_fail_buck')
plt.xlabel('Position␣along␣the␣bridge␣(mm)')
plt.ylabel('Shear␣Force␣(N)')
plt.title('Shear␣Force␣Failure␣Envelope␣(Buckling)')
plt.legend()
plt.grid(True)

# Plot M_fail_compression
plt.subplot(3, 2, 4)
plt.plot(range(1201), M_fail_compression, label='
    M_fail_compression')
plt.xlabel('Position␣along␣the␣bridge␣(mm)')
plt.ylabel('Bending␣Moment␣(N-mm)')
plt.title('Bending␣Moment␣Failure␣Envelope␣(Compression)')
plt.legend()
plt.grid(True)
```

```
# Plot M_fail_tension
plt.subplot(3, 2, 5)
plt.plot(range(1201), M_fail_tension, label='M_fail_tension')
plt.xlabel('Position along the bridge (mm)')
plt.ylabel('Bending Moment (N-mm)')
plt.title('Bending Moment Failure Envelope (Tension)')
plt.legend()
plt.grid(True)

# Plot M_fail_buck
plt.subplot(3, 2, 6)
plt.plot(range(1201), M_fail_buck, label='M_fail_buck')
plt.xlabel('Position along the bridge (mm)')
plt.ylabel('Bending Moment (N-mm)')
plt.title('Bending Moment Failure Envelope (Buckling)')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```